

南开大学信息对抗技术实验报告

实验 9：木马基本功能的实现

基于 C++/WinSock 的受控远程管理实验

学院：	密码与网络空间安全学院
学号：	2313177
姓名：	张昌源
实验环境：	Windows 10 虚拟机、本地回环网络、Visual Studio C++17
程序名称：	Remote Lab Trojan Experiment

2026 年 6 月 16 日

目录

1 实验概述	2
1.1 实验目的	2
1.2 实验环境与材料	2
2 系统设计与实验原理	3
2.1 总体架构	3
2.2 应用层报文协议	3
2.3 受控文件操作范围	4
3 程序结构与关键设计	5
3.1 源码文件分工	5
3.2 可靠收发机制	5
3.3 服务端命令分发	6
4 主要功能实现	7
4.1 输出字符串	7
4.2 定时关机与取消关机	7
4.3 获取 C 盘实验文件列表	7
4.4 截屏	8
4.5 删除、上传与下载	9
5 实验过程与结果截图	10
5.1 编译与程序启动	10
5.2 输出字符串功能测试	11
5.3 获取服务端 C 盘实验文件列表	12
5.4 截屏功能测试	13
5.5 上传与下载功能测试	13
5.6 删除功能测试	14
5.7 关机与取消关机功能测试	14
6 实验结果分析	15
6.1 功能完成情况	15
6.2 程序特点分析	16
6.3 不足与改进方向	16
7 实验总结	16

1 实验概述

1.1 实验目的

本实验围绕“木马基本功能的实现”这一教学任务，在可控虚拟机环境中实现一个客户端/服务端结构的远程控制原型程序。实验要求在设置源、目的 IP 地址后，由客户端向服务端发送指令，使服务端完成输出字符串、定时关机、取消关机、获取 C 盘文件列表、截屏、删除文件、上传文件和下载文件等操作。

通过本次实验，需要掌握以下内容：

- 1. 理解客户端/服务端模型下远程命令执行的基本流程；
- 2. 掌握 Windows 环境下基于 WinSock 的 TCP 通信方法；
- 3. 设计简单可靠的自定义应用层报文格式；
- 4. 将网络通信、文件读写、屏幕截取和系统命令调用组合成完整实验程序；
- 5. 在实验报告中结合代码和截图验证各项功能是否满足要求。

1.2 实验环境与材料

表 1: 实验环境与材料

项目	内容
操作系统	Windows 10 x64 虚拟机
开发语言	C++17
网络接口	WinSock2, TCP, IPv4, 本次实验主要使用 127.0.0.1:5050
图形界面	Win32 API, 提供 server GUI 与 client GUI
编译方式	Visual Studio 2022 MSVC, build_msvc.bat; 同时提供 CMake 构建文件
核心源码	common.hpp/cpp、server.cpp、client.cpp、server_gui.cpp、client_gui.cpp
实验目录	C:\TrojanLab\server_files, 作为受控 C 盘实验文件列表目录

本实验中使用的程序不包含自启动、隐藏驻留、权限绕过、提权等功能，所有文件操作均限制在实验目录中完成。这样既能覆盖课程要求中的“木马基本功能”，又能避免误删系统文件或影响虚拟机外部环境。

2 系统设计与实验原理

2.1 总体架构

程序采用典型的客户端/服务端结构。服务端在指定 IP 和端口监听，客户端连接后通过按钮触发不同命令。每个命令都会被封装为统一的 Packet 报文发送给服务端；服务端解析命令类型后执行对应操作，并将文本结果或文件数据返回给客户端。

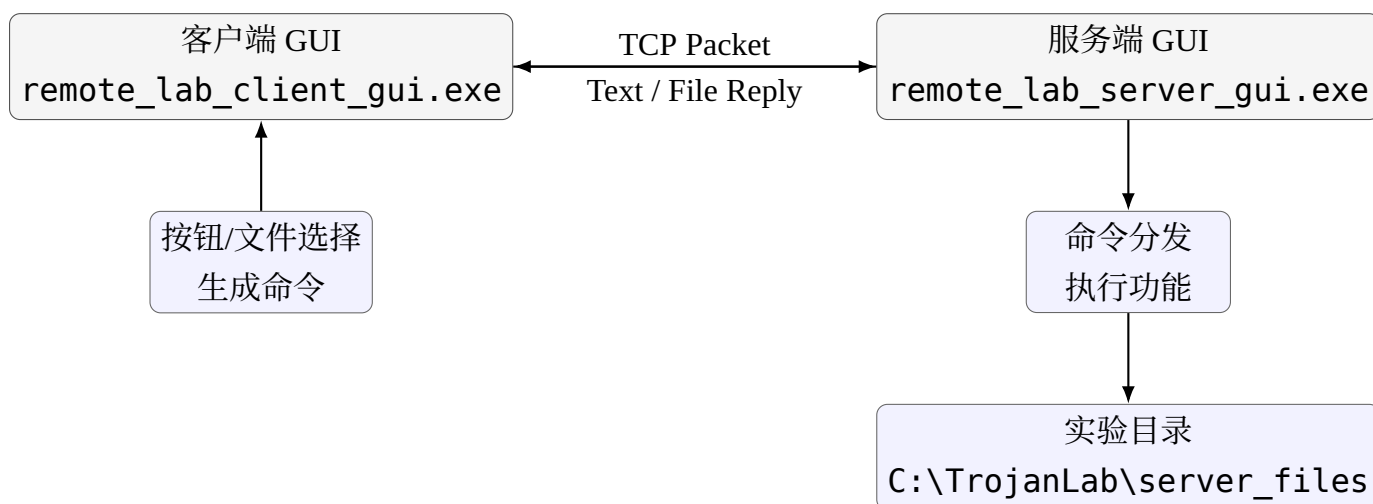


图 1: 客户端/服务端总体架构

2.2 应用层报文协议

为了避免直接发送不定长字符串造成解析困难，程序在 TCP 流上自定义了统一的 12 字节报文头。报文头由魔数、消息类型和负载长度组成，随后跟随指定长度的 payload。该设计使文本命令、文件内容和屏幕截图都可以在同一套收发函数中传输。

表 2: Packet 报文结构

字段	长度	说明
Magic	4 字节	固定为 RLB1，用于识别实验报文
Type	4 字节	大端序无符号整数，对应具体命令或返回类型
Length	4 字节	payload 字节数，最大限制为 64 MB
Payload	变长	文本、文件名、文件内容或命名文件负载

消息类型在 common.hpp 中统一定义。客户端发送 Echo、Shutdown、ListFiles 等请求，服务端返回 Ok、Error、Text 或 File。

Listing 1: 消息类型定义摘录

```
1 enum class MessageType : std::uint32_t {  
2     Echo = 1,  
3     Shutdown = 2,  
4     CancelShutdown = 3,  
5     ListFiles = 4,  
6     Screenshot = 5,  
7     DeleteFile = 6,  
8     UploadFile = 7,  
9     DownloadFile = 8,  
10    Quit = 9,  
11  
12    Ok = 100,  
13    Error = 101,  
14    Text = 102,  
15    File = 103  
16 };
```

2.3 受控文件操作范围

课程任务要求获取服务端 C 盘文件列表,并对文件进行删除、上传、下载。为了保证实验可控,本程序没有直接开放整个 C 盘,而是将 C:\TrojanLab\server_files 作为“服务端 C 盘实验文件列表”。服务端启动时会自动创建该目录,并生成 sample_server.txt 作为测试文件。

在删除和下载功能中,服务端会调用路径解析函数检查客户端传入路径。程序拒绝绝对路径和 .. 父目录引用,并通过规范化路径确认目标文件仍位于实验根目录下。这样可以满足文件操作实验需求,同时避免越界访问。

3 程序结构与关键设计

3.1 源码文件分工

表 3: 源码文件功能分工

文件	主要功能
common.hpp / common.cpp	定义消息类型、Packet 结构、WinSock 初始化类、报文收发函数、字符串/文件二进制转换函数、命名文件负载封装与解析函数。
server.cpp	命令行版服务端, 完成监听、接受连接、解析 Packet、执行各类命令。
client.cpp	命令行版客户端, 提供菜单式交互, 发送命令并接收返回结果。
server_gui.cpp	图形界面版服务端, 提供 Bind IP、Port、Root、Start/Stop、Server log 等控件。
client_gui.cpp	图形界面版客户端, 提供连接、发送字符串、关机、取消关机、刷新文件列表、截屏、上传、下载、删除等按钮。
build_msvc.bat	使用 Visual Studio 2022 编译四个可执行文件。

3.2 可靠收发机制

TCP 是面向字节流的协议, 一次 send 不一定对应一次完整 recv。因此程序在 common.cpp 中实现了 send_all 与 recv_all, 通过循环确保指定长度的数据全部发送或接收完成。随后 send_packet 先发送 12 字节头部, 再发送 payload; receive_packet 则先读取头部, 再按 length 字段读取完整负载。

Listing 2: Packet 发送函数核心逻辑摘录

```
1 bool send_packet(SOCKET socket_handle,
2                 MessageType type,
3                 const std::vector<std::uint8_t>& payload,
4                 std::string& error) {
5     if (payload.size() > kMaxPayloadSize) {
6         error = "payload is larger than 64 MB";
7         return false;
8     }
9
10    std::array<std::uint8_t, kHeaderSize> header{};
```

```
11     std::copy(kMagic.begin(), kMagic.end(), header.begin());
12     write_u32_network(header.data() + 4, static_cast<std::uint32_t>(
type));
13     write_u32_network(header.data() + 8, static_cast<std::uint32_t>(
payload.size()));
14
15     if (!send_all(socket_handle, header.data(), header.size(), error))
16     {
17         return false;
18     }
19     if (!payload.empty() && !send_all(socket_handle, payload.data(),
payload.size(), error)) {
20         return false;
21     }
22     return true;
}
```

3.3 服务端命令分发

服务端收到 Packet 后, 使用 `switch(packet.type)` 分发功能。文本类命令返回 Ok 或 Text; 截图和下载等二进制结果返回 File 类型。该方式让每个实验功能都对应明确的消息编号, 便于调试和截图验证。

Listing 3: 服务端命令处理逻辑摘录

```
1 bool handle_packet(SOCKET client, const Packet& packet, const
ServerConfig& config) {
2     std::string error;
3     switch (packet.type) {
4     case MessageType::Echo: {
5         const std::string text = remote_lab::payload_to_string(packet.
payload);
6         log_event("client says: " + text);
7         return reply_text(client, MessageType::Ok, "server printed
string: " + text);
8     }
9     case MessageType::Shutdown: {
10         const int code = std::system("shutdown /s /t 60");
11         return reply_text(client, code == 0 ? MessageType::Ok :
MessageType::Error,
12                             code == 0 ? "server host will shut down in 60
seconds"
13                             : "shutdown command failed");
}
```

```
14     }
15     case MessageType::CancelShutdown: {
16         const int code = std::system("shutdown /a");
17         return reply_text(client, code == 0 ? MessageType::Ok :
18             MessageType::Error,
19             code == 0 ? "pending shutdown was cancelled"
20                 : "cancel shutdown command failed");
21     };
22     case MessageType::ListFiles:
23         return reply_text(client, MessageType::Text, list_files(config.
24             root));
25     default:
26         return reply_text(client, MessageType::Error, "unknown command
27             type");
28     }
29 }
```

4 主要功能实现

4.1 输出字符串

客户端在字符串输入框中输入文本, 点击 **Send string** 后发送 Echo 消息。服务端将收到的字符串写入日志, 并返回确认文本。该功能用于验证连接建立、文本编码和基本命令往返过程是否正常。

4.2 定时关机与取消关机

关机功能由客户端发送 Shutdown 命令触发, 服务端调用 Windows 系统命令 `shutdown /s /t 60`, 使服务端主机在 60 秒后关机。取消关机功能发送 CancelShutdown 命令, 服务端调用 `shutdown /a` 取消尚未执行的关机计划。

这两个功能直接操作系统状态, 因此本次实验只在 Windows 10 虚拟机中测试, 并在 60 秒倒计时内立即执行取消命令。

4.3 获取 C 盘实验文件列表

服务端通过 `std::filesystem::directory_iterator` 遍历实验目录, 将文件类型、大小和相对路径格式化为文本返回。客户端收到结果后, 一方面将原始结果写入日志, 另一方面从文本中提取文件名并刷新左侧 **Server files** 列表。

Listing 4: 服务端文件列表生成逻辑摘录

```
1 std::string list_files(const std::filesystem::path& root) {
2     std::ostream oss;
3     oss << "Experiment C drive root: " << remote_lab::narrow_path(root)
4         << "\n";
5     oss << "Use the RelativePath column for delete/download commands.\n\n";
6     oss << std::left << std::setw(8) << "Type"
7         << std::setw(14) << "Size" << "RelativePath\n";
8     for (const auto& entry : std::filesystem::directory_iterator(root))
9     {
10         const bool is_dir = entry.is_directory();
11         const bool is_file = entry.is_regular_file();
12         std::uintmax_t size = is_file ? entry.file_size() : 0;
13         oss << std::left << std::setw(8) << (is_dir ? "DIR" : "FILE")
14             << std::setw(14) << (is_dir ? "-" : std::to_string(size))
15             << remote_lab::narrow_path(entry.path().filename()) << "\n";
16     }
17     return oss.str();
18 }
```

4.4 截屏

服务端使用 Windows GDI 接口完成桌面截取。程序首先获取屏幕 DC, 创建兼容 DC 和位图, 然后通过 `BitBlt` 将屏幕图像复制到内存位图中, 再通过 `GetDIBits` 取出像素数据并组装为 BMP 文件。服务端将 BMP 作为 `File` 类型报文返回, 客户端弹出保存对话框保存为 `server_screen.bmp`。

Listing 5: 截屏核心逻辑摘录

```
1 bool capture_screen_bmp(std::vector<std::uint8_t>& output, std::string&
2     error) {
3     HDC screen_dc = GetDC(nullptr);
4     const int width = GetSystemMetrics(SM_CXSCREEN);
5     const int height = GetSystemMetrics(SM_CYSCREEN);
6     HDC memory_dc = CreateCompatibleDC(screen_dc);
7     HBITMAP bitmap = CreateCompatibleBitmap(screen_dc, width, height);
8     HGDIOBJ old_object = SelectObject(memory_dc, bitmap);
9     if (!BitBlt(memory_dc, 0, 0, width, height, screen_dc, 0, 0,
```

```
SRCCOPY)) {  
    error = "BitBlt failed";  
    return false;  
}  
  
BITMAPINFOHEADER info{};  
info.biSize = sizeof(BITMAPINFOHEADER);  
info.biWidth = width;  
info.biHeight = height;  
info.biPlanes = 1;  
info.biBitCount = 32;  
info.biCompression = BI_RGB;  
  
std::vector<std::uint8_t> pixels(width * height * 4);  
GetDIBits(memory_dc, bitmap, 0, static_cast<UINT>(height),  
    pixels.data(), reinterpret_cast<BITMAPINFO*>(&info),  
    DIB_RGB_COLORS);  
// 之后将 BMP 文件头、DIB 头和像素数据拼接为 output  
return true;  
}
```

4.5 删除、上传与下载

删除功能要求用户先从服务端文件列表中选择目标文件,客户端发送相对路径,服务端通过 `resolve_safe_path` 检查路径合法性后调用 `std::filesystem::remove` 删除文件。

上传功能由客户端通过文件选择框读取本地文件内容,封装为命名 `payload` 后发送给服务端;服务端不直接使用原文件名,而是统一保存为实验要求中的 `myFile.txt`。下载功能则相反,客户端选择服务端文件,服务端读取该文件内容并返回命名文件负载,客户端保存为 `myFile.txt`。

Listing 6: 路径限制与上传/下载核心逻辑摘录

```
1 bool resolve_safe_path(const std::filesystem::path& root,  
2                       const std::string& relative_text,  
3                       std::filesystem::path& resolved,  
4                       std::string& error) {  
5     std::filesystem::path relative(relative_text);  
6     if (relative.empty() || relative.is_absolute()) {  
7         error = "absolute paths are not accepted";  
8         return false;  
9     }
```

```
10     for (const auto& part : relative) {
11         if (part == "..") {
12             error = "parent directory references are not accepted";
13             return false;
14         }
15     }
16     // 规范化 root 与目标路径, 并检查目标仍在 root 内
17     return true;
18 }
19
20 case MessageType::UploadFile: {
21     std::string original_name;
22     std::vector<std::uint8_t> data;
23     parse_named_payload(packet.payload, original_name, data, error);
24     const auto output_path = config.root / "myFile.txt";
25     write_binary_file(output_path, data, error);
26     return reply_text(client, MessageType::Ok,
27         "uploaded content saved to server file myFile.txt
28 ");
29 }
30
31 case MessageType::DownloadFile: {
32     const std::string relative = payload_to_string(packet.payload);
33     std::filesystem::path target;
34     resolve_safe_path(config.root, relative, target, error);
35     std::vector<std::uint8_t> data;
36     read_binary_file(target, data, error);
37     return send_packet(client, MessageType::File,
38         make_named_payload("myFile.txt", data), error);
39 }
```

5 实验过程与结果截图

5.1 编译与程序启动

使用 build_msvc.bat 编译后, 生成了命令行和图形界面两个版本的客户端、服务端可执行文件。实验主要使用 GUI 版本进行演示。

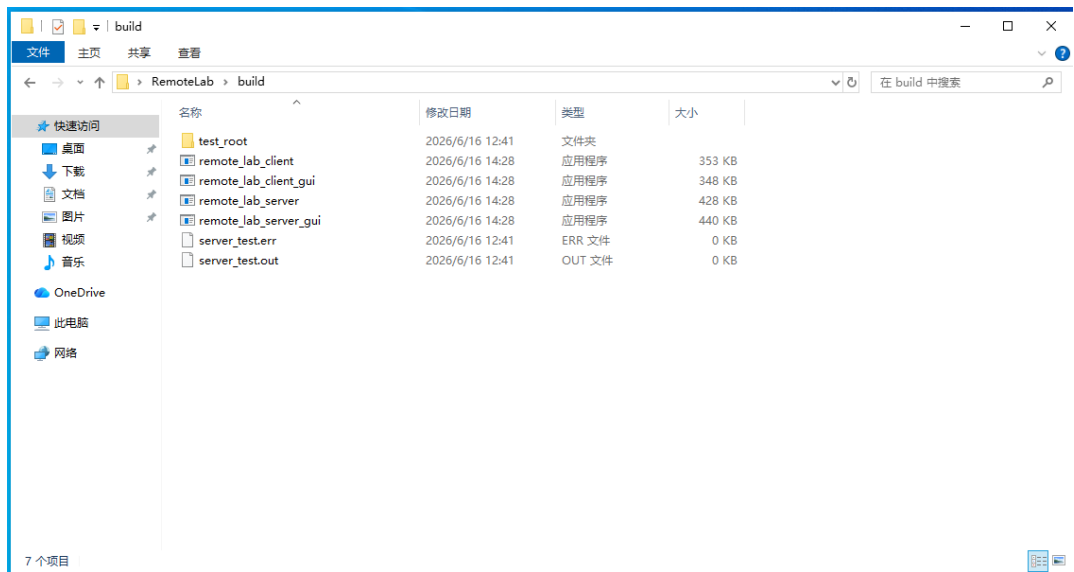


图 2: 编译后生成的客户端和服务端程序

服务端绑定 127.0.0.1、端口 5050,实验根目录设置为 C:\TrojanLab\server_files。客户端连接同一地址和端口后,服务端日志显示 Client connected: 127.0.0.1,说明 TCP 连接建立成功。



图 3: 服务端监听与客户端连接

5.2 输出字符串功能测试

在客户端输入 hello server 并点击 Send string。客户端日志显示 server printed string: hello server,服务端日志同步出现 Output string: hello server,说明字符串发送、服务端输出和返回确认均正常。



图 4: 输出字符串功能测试

5.3 获取服务端 C 盘实验文件列表

点击 Refresh file list 后，客户端日志中显示实验目录路径、文件类型、大小和相对路径，左侧 Server files 列表中同步出现 sample_server.txt。这说明服务端文件列表能够被客户端成功获取和解析。

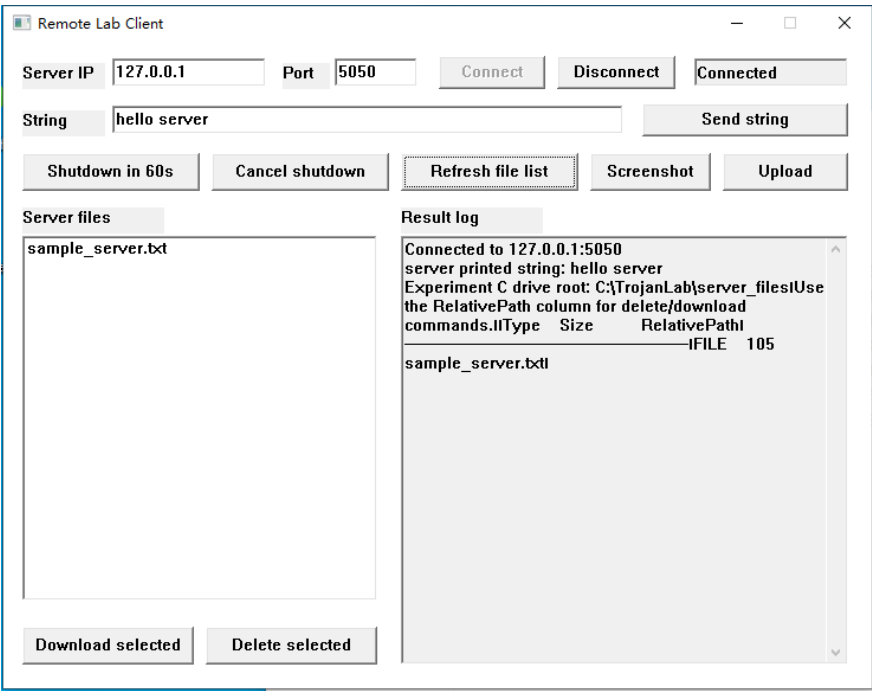


图 5: 获取服务端实验文件列表

5.4 截屏功能测试

点击 Screenshot 后, 客户端弹出保存对话框, 默认文件名为 server_screen。保存完成后打开 server_screen.bmp, 可以看到图像内容为服务端桌面和正在运行的实验窗口, 证明服务端桌面图像已经成功截取并传回客户端。

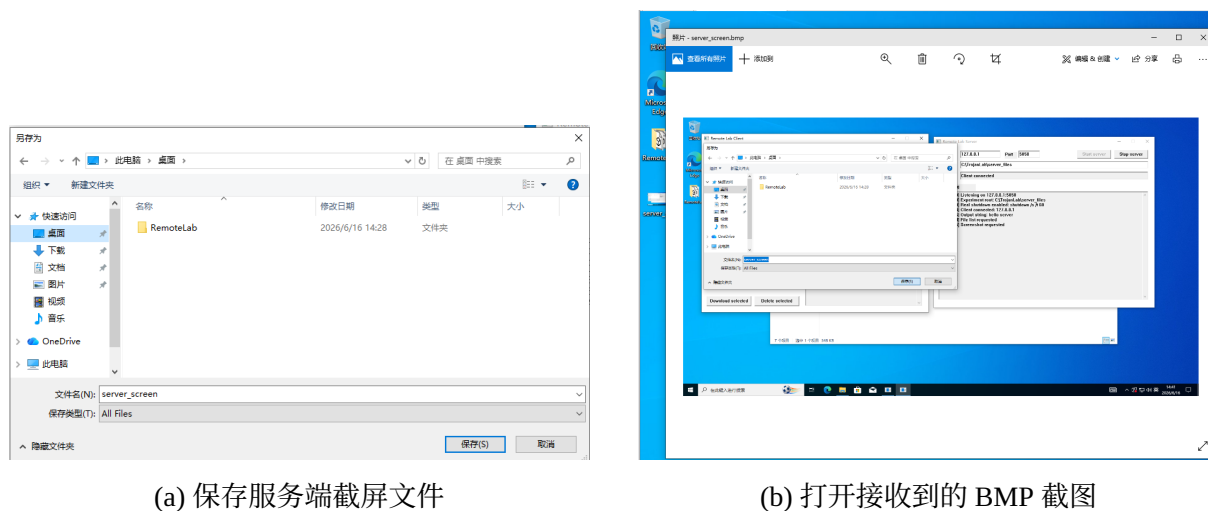


图 6: 截屏功能测试

5.5 上传与下载功能测试

上传测试中, 客户端选择本地 upload.txt 后, 服务端将其保存为 myFile.txt。刷新文件列表后, 客户端左侧出现 myFile.txt 和 sample_server.txt 两个文件。下载测试中, 客户端选择 myFile.txt 并保存到桌面, 打开文件可以看到内容为 This is a test message!, 说明文件内容传输正确。

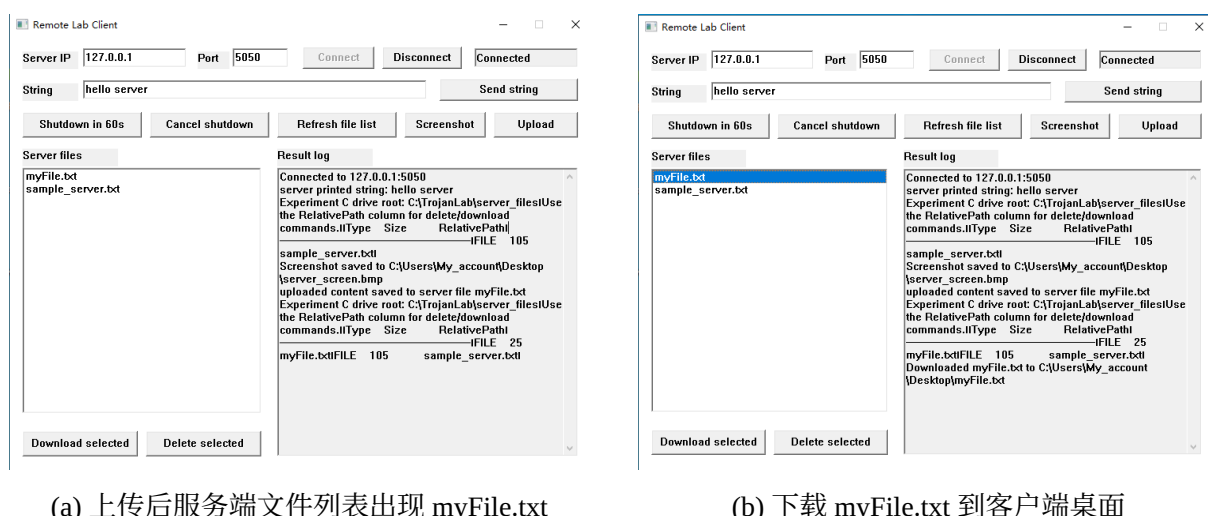


图 7: 上传与下载操作记录

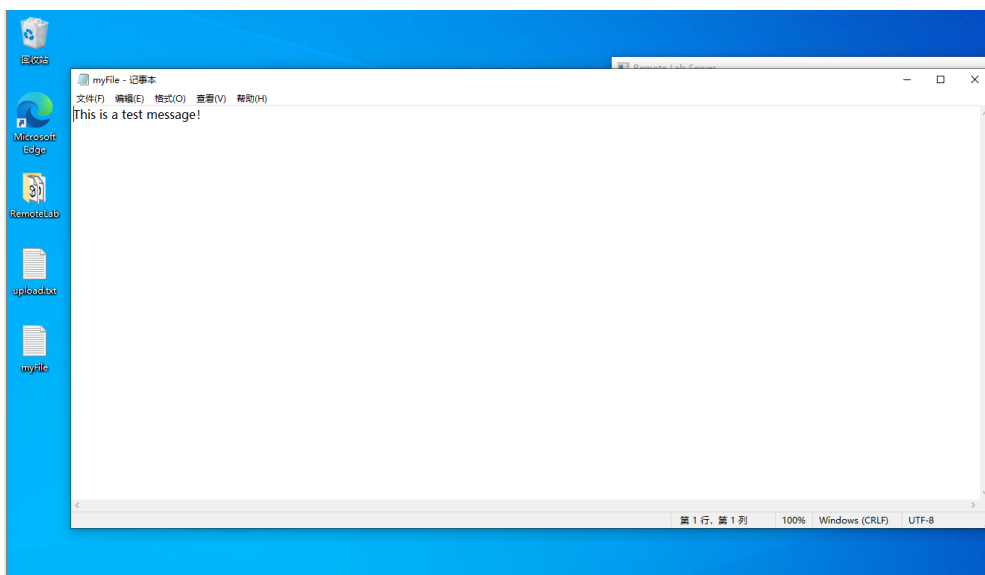
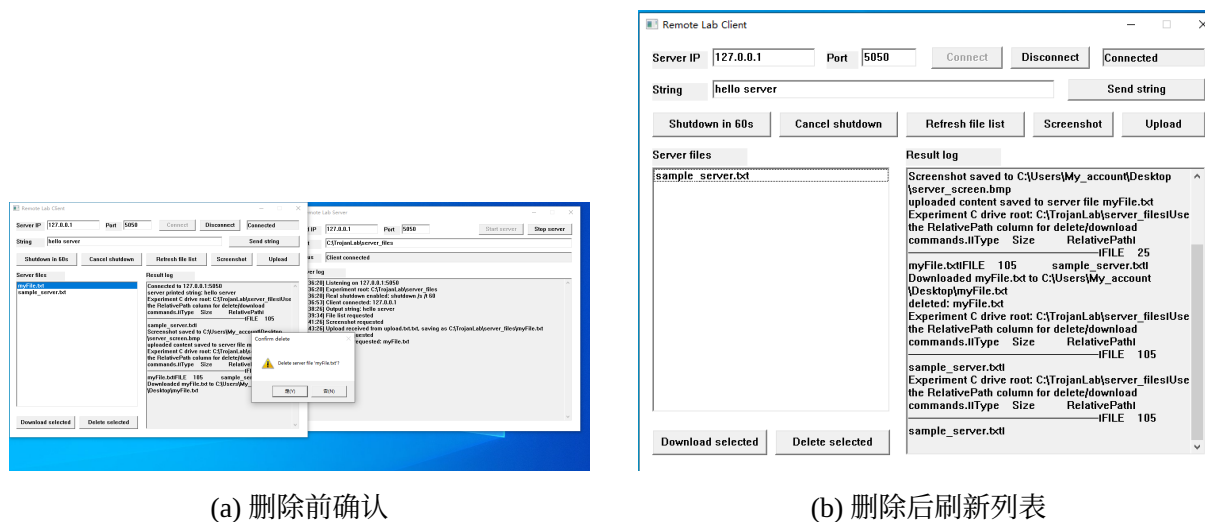


图 8: 下载文件内容验证

5.6 删除功能测试

选择服务端文件列表中的 myFile.txt 后点击 Delete selected, 程序弹出确认对话框, 避免误操作。确认删除后再次刷新文件列表, myFile.txt 消失, 仅剩 sample_server.txt, 说明删除功能执行成功。



(a) 删除前确认

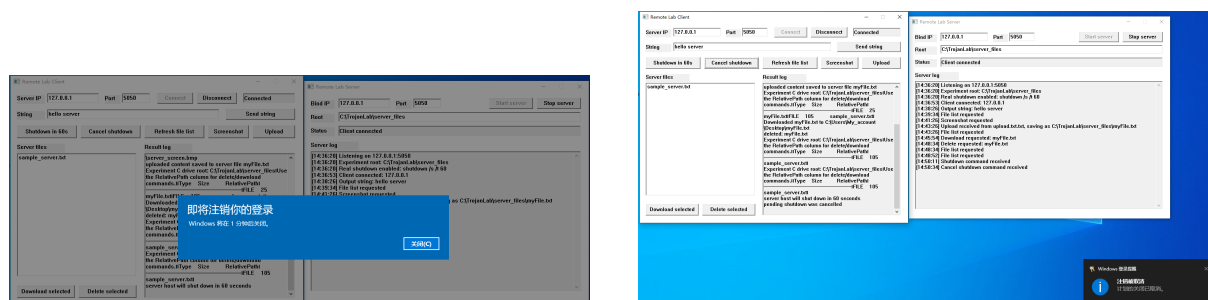
(b) 删除后刷新列表

图 9: 删除服务端文件功能测试

5.7 关机与取消关机功能测试

点击 Shutdown in 60s 后, Windows 弹出“即将注销你的登录 / Windows 将在 1 分钟后关闭”的系统提示, 同时客户端日志显示 server host will shut down in 60 seconds。随后在 60 秒内点击 Cancel shutdown, 右下角出现“注销被取

消”的提示, 客户端日志显示 `pending shutdown was cancelled`, 服务端日志也记录了关机 and 取消关机命令。



(a) 服务端触发 60 秒关机提示

(b) 取消关机成功

图 10: 关机与取消关机功能测试

6 实验结果分析

6.1 功能完成情况

表 4: 实验功能完成情况汇总

序号	实验要求	实现与验证方式	结果
1	输出字符串	客户端发送 <code>Echo</code> , 服务端日志输出字符串并返回确认信息。	完成
2	关机	服务端接收 <code>Shutdown</code> 后执行 60 秒定时关机命令, 系统弹出 1 分钟关机提示。	完成
3	取消关机	服务端接收 <code>CancelShutdown</code> 后取消未执行的关机计划, 系统提示关机计划已取消。	完成
4	获取 C 盘文件列表	服务端遍历实验目录, 并返回文件类型、文件大小和相对路径。	完成
5	截屏	服务端用 GDI 截取桌面为 BMP, 客户端保存并成功打开。	完成
6	删除	客户端选择文件后发送相对路径, 服务端安全解析路径并删除目标文件。	完成
7	上传	客户端选择本地文件, 服务端保存为 <code>myFile.txt</code> , 刷新列表可见。	完成
8	下载	客户端选择服务端文件, 接收后保存为 <code>myFile.txt</code> , 打开内容正确。	完成

6.2 程序特点分析

本程序的主要特点如下:

1. **协议清晰。**所有请求和响应都使用统一 Packet 格式, 消息类型和负载长度明确, 便于扩展和调试。
2. **二进制传输完整。**截图和文件下载均使用 `std::vector<std::uint8_t>` 保存原始字节, 避免文本编码影响文件内容。
3. **界面操作直观。**GUI 客户端将实验功能映射为按钮, 服务端和客户端均有日志窗口, 截图证据较完整。
4. **文件操作可控。**程序将文件列表、删除、上传、下载限制在实验目录中, 降低误操作风险。
5. **具备命令行备用版本。**除 GUI 程序外, 项目也包含命令行版客户端和服务端, 便于在不同实验环境下测试。

6.3 不足与改进方向

本实验已经完成课程要求的基本功能, 但仍存在一些可以改进的地方:

1. 当前协议没有身份认证机制, 若用于真实环境, 至少应加入口令认证或会话令牌;
2. 当前通信没有加密, 实验环境中可以观察明文命令和文件内容, 后续可加入 TLS 或对称加密;
3. 当前服务端一次主要处理一个客户端连接, 若要支持多客户端, 可增加线程池或异步 I/O;
4. 当前文件列表只展示实验目录的一层文件, 后续可以加入递归目录浏览和目录区分显示;
5. 当前 GUI 以日志文本展示结果, 后续可以对文件列表、下载状态、错误信息做更结构化的显示。

7 实验总结

本次实验基于 C++17 和 WinSock 实现了一个受控远程管理程序, 完整覆盖了课程要求中的八项功能。实验过程中, 客户端负责构造命令和展示结果, 服务端负责监听连接、解析 Packet、执行对应系统或文件操作, 并将结果返回。通过截图可以看到, 字符串输出、定时关机、取消关机、文件列表获取、桌面截屏、文件删除、上传和下载均已成功运行。

从实现角度看, 本实验的关键在于把不同类型的功能统一到同一个网络协议框架中: 文本命令使用字符串 payload, 文件与截图使用命名二进制 payload, 服务端通过消息类型进行分发。实验也让我进一步理解了远程控制类程序的基本组成: 网络连接、命令协议、系统调用、文件管理和结果回传。同时, 程序通过限制实验目录和使用虚拟机

测试, 使实验过程保持在可控范围内。

总体而言, 本次实验不仅完成了木马基本功能的课程验证, 也帮助我更直观地理解了远程控制程序的工作机制、潜在风险和必要的边界控制。